



GUIA DA AEP: 4º Semestre (2026.2)

Engenharia de Software (Esoft) e Análise e Desenv. de Sistemas (ADS).

1. O Propósito do Projeto

Neste semestre, as turmas de Esoft e ADS trabalharão no mesmo grande desafio. Vocês formarão equipes (Squads de até 3 pessoas) para criar uma **Solução Computacional**.

A única regra de negócio inegociável: O seu software deve resolver um problema real ligado a um dos **Objetivos de Desenvolvimento Sustentável (ODS) da ONU**. Vocês vão projetar e programar algo que gere impacto positivo no mundo real.

2. Jogo Limpo: O que será cobrado do meu curso?

Sabemos que Esoft e ADS possuem grades curriculares diferentes. Fiquem tranquilos: **a avaliação respeitará o foco técnico do seu respectivo curso.**

-  **O QUE É COMUM A TODOS (Esoft e ADS):**
 - Todos devem programar usando os Pilares da Orientação a Objetos (POO).
 - Todos devem persistir os dados em um Banco de Dados real.
 - Todos devem usar versionamento de código no GitHub.
-  **O FOCO DA ENGENHARIA DE SOFTWARE (ESOFT):**
 - O foco está no rigor arquitetural e na **Engenharia de Requisitos**. O "Cronograma" exigido para vocês não é um gráfico complexo de gestão, é simplesmente a estruturação de um **Backlog Ágil** (Planejar de forma simples quem vai programar qual funcionalidade e em qual momento).
-  **O FOCO DE ANÁLISE E DESENV. DE SISTEMAS (ADS):**
 - O foco engloba o aprofundamento na **Gestão de Projetos** (gerenciamento de escopo, prazos e viabilidade) e a aplicação dos conceitos de usabilidade em **Interfaces (ETI)**. O planejamento de vocês deve refletir o controle do projeto e a jornada visual do usuário.

3. O CHECKLIST DE ENTREGAS E NOTAS

Entreguem os itens solicitados para garantir a pontuação (Valor Total: 1,0 Ponto por Bimestre).

ENTREGA 1: Planejamento, Escopo e Estruturação (Final do 1º Bimestre)

O que entregar: Um documento PDF formal e o link do Repositório do GitHub. *Não é necessária a entrega de código funcional nesta fase.*

O que deve ter no PDF / GitHub? (Checklist)	Critério de Aceitação para a Nota	Peso
1. O Problema e os Requisitos (Escopo) Descreva qual problema da ODS da ONU vocês vão resolver. Escreva uma lista com no mínimo 5 Requisitos Funcionais . (Ex: O sistema deve permitir o cadastro de doadores contendo nome, idade, CPF, e-mail e tipo sanguíneo).	A dor (problema) está clara? Os requisitos listados descrevem o que o sistema fará para atender à necessidade do usuário de forma direta e sem jargões confusos?	0,2
2. O Planejamento (Cronograma/ Backlog) Uma tabela inserida no PDF contendo as etapas de desenvolvimento do 2º bimestre (Datas, Atividades/Stories e Responsável).	A equipe demonstrou capacidade de autogestão estruturando o trabalho futuro na tabela?	0,2
3. A Justificativa Técnica e Visual Um texto defendendo as escolhas do projeto. Por que escolheram essa Linguagem de Programação? Por que esse Banco de Dados? Se for ADS, justifique as escolhas de interface.	A equipe explicou tecnicamente os motivos das escolhas e como elas suportam o Escopo definido no item 1?	0,3
4. Diagramas e GitHub Estruturado O PDF deve conter o Link Público do GitHub. O repositório deve ter a estrutura de pastas criada (<code>/src</code> , <code>/docs</code> , <code>/database</code>). No PDF devem constar o Diagrama de Classes e o Diagrama do Banco (DER) .	O link do GitHub está acessível? O Diagrama de Classes atende aos requisitos mínimos (Composição 1:N, Herança e Polimorfismo)?	0,3

 **Atenção:** Verifiquem se o link do GitHub está configurado como PÚBLICO. Links quebrados ou privados impedem a correção e avaliação do projeto.

📌 ENTREGA 2: Implementação, Rigor e Integração (Final do 2º Bimestre)

O que entregar: O Código-Fonte completo da aplicação e o Script do Banco de Dados hospedados no GitHub.

O que o Código deve fazer? (Checklist)	Critério de Aceitação para a Nota	Peso
1. Aderência ao Planejamento O software entregue deve refletir as funcionalidades prometidas no escopo da 1ª entrega.	O sistema faz sentido com a proposta original? Alterações de escopo são normais, mas mudar 100% do projeto sem justificativa reduz este ponto.	0,2
2. Pilares de POO O código deve ser tipado, seguir convenções de nomenclatura (<i>CamelCase</i>) e aplicar o paradigma de Orientação a Objetos.	O código implementa de forma clara: 1) Classe Abstrata ou Interface; 2) Herança; e 3) Polimorfismo via <code>@Override</code> ?	0,3
3. Operações CRUD Funcionais O núcleo do sistema precisa estar operante sobre a entidade principal do projeto.	O software consegue realizar operações de Create, Read, Update e Delete no banco de forma funcional?	0,3
4. Persistência em Banco de Dados O sistema tem que salvar as informações em um Banco de Dados real (relacional ou NoSQL). O script de criação das tabelas (ex: arquivo <code>.sql</code>) deve estar na pasta <code>database</code> do GitHub.	A conexão com o banco de dados ocorre com sucesso? (Simular o banco de dados utilizando apenas listas temporárias na memória RAM invalida este critério).	0,2

4. ORIENTAÇÕES FINAIS PARA O DESENVOLVIMENTO

1. A Interface (Tela) vs. Arquitetura (A diferença entre os cursos):

- **Para Esoft:** Uma aplicação rodando perfeitamente via Terminal/Console com uma boa arquitetura vale muito mais do que um sistema visualmente lindo que não persiste dados.
- **Para ADS:** O design e a usabilidade importam! Usem os conhecimentos de ETI para criar telas funcionais e amigáveis, mas garantam que elas estejam conectadas ao Banco de Dados e respeitem a POO.

2. Ambiente e Execução: O repositório do projeto no GitHub deve conter um arquivo `README.md` com instruções curtas e claras de como o professor deve rodar o seu software (como instalar dependências, rodar o banco, etc), respeitando a compatibilidade de ambientes.

3. **Trabalho em Equipe:** O GitHub registra o histórico de commits. A evolução do código no repositório deve refletir a participação e o esforço conjunto de todos os integrantes do Squad.
4. **Originalidade Acadêmica:** Usem a criatividade de vocês! Projetos com estrutura de classes, nomes de variáveis e lógicas integralmente copiadas de outros grupos configuram plágio e inviabilizam a avaliação.

Use a AEP como o laboratório seguro para testar, errar, debater e construir um projeto que dará orgulho de colocar no portfólio profissional de vocês. Bom trabalho a todos!

5. FAQ - PERGUNTAS FREQUENTES Esoft & ADS

1. Professor, o meu grupo é apenas de Engenharia (Esoft). Precisamos fazer uma interface (tela) complexa para não perder nota? E se o grupo for de ADS?

A avaliação respeitará a grade do seu curso.

- **Para ESOFT:** Como vocês não possuem disciplinas de Interface neste semestre, a estética da tela não será avaliada. Um sistema rodando perfeitamente via Terminal/Console que atenda aos pilares da POO e Banco de Dados pode tirar nota máxima. Foquem na arquitetura (o "motor" do sistema) e nos Requisitos.
- **Para ADS:** Como vocês possuem a disciplina de ETI (Estruturação de Telas/Interfaces), espera-se que o projeto possua uma interface gráfica minimamente navegável que reflita uma boa usabilidade. Além disso, a capacidade de Planejamento (Gestão de Projetos) será mais observada na estruturação do Cronograma do grupo.

2. Posso fazer o trabalho sozinho ou o grupo de 3 pessoas é obrigatório?

É altamente recomendável que seja feito em equipes de até 3 pessoas para treinar o trabalho colaborativo (uso do Git, divisão de tarefas e integração de módulos). Caso haja uma necessidade extrema, o trabalho pode ser feito individualmente, mas o nível de exigência da avaliação (UML, POO, Banco de Dados) será o mesmo para a entrega.

3. Na 2ª Entrega (Código), é OBRIGATÓRIO usar Herança e Polimorfismo. E se a ideia do nosso projeto for muito simples e não precisar disso?

A AEP é um instrumento de avaliação acadêmica. Precisamos medir se vocês dominam a Orientação a Objetos na prática. Portanto, o planejamento do sistema (1º Bimestre) já deve ser desenhado com uma complexidade que exija Herança e Polimorfismo. Pensem, por exemplo, em criar diferentes tipos de perfis de usuário (Admin, Doador, Cliente) que herdam de uma classe "Pessoa", ou tipos de doação diferentes. Adaptem o problema de negócio à exigência técnica.

4. Nós fazemos chamadas no Discord e programamos juntos na tela de um colega (Pair Programming). Tem problema se só um aluno fizer o commit (envio) do código no GitHub?

Sim. O GitHub registra o histórico de participação. Mesmo programando juntos, vocês devem dividir o trabalho no repositório (ex: um faz o commit do Banco, outro faz o das Classes, outro constrói a Tela, outro faz o README). O avaliador precisa da prova técnica (o histórico do Git) de que **todos** os alunos da Squad sabem utilizar a ferramenta de controle de versão.

5. No 1º Bimestre precisamos montar um Cronograma/Backlog. Mas no desenvolvimento (seja Ágil ou Tradicional) o escopo pode mudar. E se não der tempo de fazer um dos 5 requisitos no 2º Bimestre? Vamos perder o ponto de "Aderência"?

Os processos de software abraçam mudanças, mas não abraçam o caos. Se vocês precisarem alterar/remover uma funcionalidade no meio do caminho porque encontraram uma solução técnica melhor, **basta atualizar a documentação no README**. O que zera o critério de "Aderência" é o grupo abandonar o planejamento e entregar, no 2º semestre, um projeto de "Pizzaria" sendo que haviam aprovado a modelagem de um "Hospital" no 1º semestre. Aderência é sobre manter a coerência da solução.

6. O que significa "Persistência em Banco de Dados"? Salvar em um arquivo de texto (.txt) ou listas (Arrays) conta ponto?

Não. Salvar dados em Arrays, Listas temporárias na Memória RAM ou blocos de notas (.txt) zerará o critério de Persistência em Banco de Dados. O sistema deve se conectar a um SGDB real (MySQL, PostgreSQL, SQL Server, MongoDB, etc.) e realizar as operações via código corretamente.

7. Nosso grupo copiou um código pronto do GitHub de outro projeto da internet, mas nós mudamos o nome das variáveis. Isso pode gerar problema?

Sim. Nossas ferramentas automatizadas de avaliação realizam varreduras de Similaridade Semântica e Estrutural em lote. Alterar o nome de variáveis (de `cliente` para `usuario`) em um projeto clonado é facilmente detectável. Projetos identificados como plágio ou compra de software pronto serão zerados automaticamente, sem possibilidade de recurso. Usem a criatividade de vocês!

8. Preciso entregar código funcionando agora na entrega do 1º Bimestre?

Não. A entrega do 1º Bimestre é o "Blueprint" (A planta baixa do sistema). Vocês entregarão **apenas o documento PDF** com os Requisitos, Justificativas e os Diagramas (UML e DER), juntamente com o link do GitHub, que deve conter as pastas criadas e vazias (preparadas para receber o código do próximo bimestre).

9. E se o professor não conseguir rodar o meu código no computador dele na hora de dar a nota do 2º Bimestre?

O projeto de vocês DEVE conter um arquivo chamado `README.md` na raiz do repositório. Nesse arquivo, vocês devem escrever um passo-a-passo simples e claro ensinando o avaliador a compilar/rodar o sistema de vocês localmente (ex: "Qual versão do Java usar", "Como rodar o script SQL no banco", "Quais bibliotecas instalar"). Se o código não compilar por falta de instrução da equipe, a nota sofrerá reduções. O ambiente de execução é responsabilidade total da equipe.